



PRÁCTICA 1

LABORATORIO 1

GIT FUNDAMENTALS

Administración de Bases de Datos

ESCRIBE AQUÍ TU NOMBRE Y APELLIDOS

Escribe aquí el nombre de tu Repositorio Github

Profesor

Dr. Richard Avilés López

Grado en Ingeniería Informática

GIT PROFESIONAL PARA SOFTWARE ENGINEERING, QA Y ORACLE DBA*Serie de laboratorios — Bloque: Control de versiones profesional*

LABORATORIO 1

GIT FUNDAMENTALS

Del directorio vacío a commit, branch, push y pull

Ficha del laboratorio	
Duración estimada	2 – 3 horas (sesión de laboratorio presencial)
Modalidad	Individual, con simulación de un segundo desarrollador para provocar un conflicto
Requisito previo	Ninguno — este es el laboratorio de entrada a la asignatura
Entregable	Repositorio en GitHub + capturas solicitadas en la sección 8 (Checklist de entrega)
Precede a	Lab 2 — GitHub Team Workflow (Issue → Branch → PR → Review → Merge)

Índice

Índice.....	3
1. Objetivos de aprendizaje	4
2. Antes de empezar — Prerrequisitos	4
3. Recordatorio conceptual rápido	5
4. Paso a paso	5
Parte A — Crear el proyecto.....	5
Parte B — Inicializar Git (git init).....	5
Parte C — Crear una estructura profesional.....	6
Parte D — README y primer commit.....	7
Parte E — Equivocarse deliberadamente (y corregirlo con seguridad)	9
Parte F — Leer el historial	10
Parte G — Branches: crear, cambiar, comprobar que NO es una carpeta	11
Parte H — Provocar y resolver un conflicto.....	12
Parte I — Conectar con GitHub	14
5. Tabla consolidada de errores frecuentes	17
6. Buenas prácticas de la industria (resumen)	18
7. Preguntas de comprobación	18
8. Checklist de entrega	19
9. Rúbrica de evaluación	19
10. Glosario rápido de este laboratorio.....	20
11. Cierre del laboratorio	20

1. Objetivos de aprendizaje

Al finalizar este laboratorio, el estudiante será capaz de:

- Verificar y configurar Git correctamente en su máquina (identidad de autor).
- Crear un repositorio desde cero y explicar qué hace exactamente `git init`.
- Diseñar una estructura de carpetas profesional para un proyecto real.
- Interpretar correctamente la salida de `git status` en cualquier situación.
- Utilizar la Staging Area de forma consciente (`git add`, `git diff`, `git diff --staged`).
- Crear commits atómicos y bien redactados siguiendo Conventional Commits.
- Cometer un error típico de principiante, diagnosticarlo y corregirlo con seguridad.
- Leer e interpretar el historial (`git log` en sus variantes más útiles).
- Crear, cambiar y eliminar branches, entendiendo que una branch NO es una carpeta.
- Provocar un conflicto de fusión de forma controlada y resolverlo sin pánico.
- Conectar un repositorio local con GitHub y realizar `push` y `pull` correctamente.
- Explicar con sus propias palabras la diferencia entre Git y GitHub.

2. Antes de empezar — Prerrequisitos

Antes de tocar un solo comando de trabajo, el estudiante debe verificar que su entorno está listo. Esto evita perder los primeros 20 minutos de laboratorio en problemas de instalación.

Requisito	Comando de verificación	Resultado esperado
Git instalado	<code>git --version</code>	git version 2.x.x (cualquier versión reciente es válida)
Identidad de autor configurada	<code>git config --global user.name</code> <code>git config --global user.email</code>	Debe devolver tu nombre y tu correo, no estar vacío
Cuenta de GitHub activa	Iniciar sesión en github.com	Acceso al panel de usuario
Editor de texto o IDE	Abrir VS Code (o el editor elegido)	El editor abre sin errores
Terminal disponible	Abrir terminal / PowerShell / bash	Prompt de comandos operativo

Verificación de entorno — ejecutar antes de empezar

```
# Si git --version falla, instala Git antes de continuar
git --version

# Configura tu identidad UNA SOLA VEZ por máquina (se guarda globalmente)
git config --global user.name "Nombre Apellido"
git config --global user.email "tu-correo@ejemplo.com"

# Verifica que ha quedado guardado
git config --global user.name
git config --global user.email
```

⚠ ERROR FRECUENTE — "funciona en mi máquina pero el commit sale con otro nombre"

Si nunca configuraste `user.name` / `user.email`, Git puede usar valores por defecto del sistema operativo (a veces vacíos o incorrectos). Cada commit queda firmado con esos datos para siempre en el historial. Configúralo ANTES del primer commit, no después.

3. Recordatorio conceptual rápido

Antes de escribir comandos, recordamos las cuatro zonas por las que viaja un cambio en Git. Este mapa mental es la base de todo lo que haremos hoy:

Las cuatro zonas de Git

Working Directory	--git add-->	Staging Area	--git commit-->	Local Repository
--git push-->	Remote Repository (GitHub)			
(escritorio)		(bandeja)		(archivo
histórico)		(oficina central)		

Idea clave que repetiremos durante todo el curso: una branch (rama) NO es una carpeta física. Es una línea de evolución del historial de commits. Cuando creamos una branch, el contenido del disco no se duplica en ninguna carpeta nueva; lo que cambia es a qué commit apunta nuestro puntero de trabajo (HEAD). Lo comprobaremos con nuestros propios ojos en la Parte G.

4. Paso a paso

Parte A — Crear el proyecto

Vamos a actuar como si acabáramos de incorporarnos a una empresa y tuviéramos que arrancar un repositorio nuevo. Nuestro proyecto se llamará oracle-database-lab.

Terminal

```
mkdir oracle-database-lab
cd oracle-database-lab
ls -la
```

En este punto tenemos una carpeta vacía en el sistema de archivos. Git todavía NO existe aquí. `ls -la` no debería mostrar ninguna carpeta .git.

Parte B — Inicializar Git (git init)

Terminal

```
git init
```

Ejecuta de nuevo `ls -la`. Ahora aparece una carpeta oculta .git. Esa carpeta es la infraestructura interna de Git: aquí vivirán todos los commits, branches y configuración del repositorio.

Terminal

```
ls -la
git status
```

`git status` debería indicar que estás en la branch principal (main) y que no hay commits todavía ("No commits yet").

⚠ ERROR FRECUENTE — Modificar manualmente la carpeta .git

Nunca edites archivos dentro de .git a mano. Es la base de datos interna de Git. Toda interacción debe hacerse mediante comandos `git ...`.

⚠ ERROR FRECUENTE — Repositorio dentro de otro repositorio

Ejecutar `git init` dentro de una carpeta que ya está dentro de otro repositorio Git crea un "repositorio anidado" que confunde a Git sobre qué .git debe usar. Antes de hacer `git init`, comprueba con `git status` si ya estás dentro de un repositorio existente.

Parte C — Crear una estructura profesional

Un repositorio real no es solo código: también organiza conocimiento técnico (scripts, documentación, procedimientos operativos). Vamos a crear la siguiente estructura:

Estructura objetivo

```
oracle-database-lab/
├── database/
│   ├── schema/
│   ├── migrations/
│   ├── rollback/
│   └── tests/
├── scripts/
├── runbooks/
├── playbooks/
├── docs/
└── README.md
```

Terminal

```
mkdir -p database/schema database/migrations database/rollback database/tests
mkdir -p scripts runbooks playbooks docs
touch README.md
git status
```

Observa el resultado de `git status`: probablemente NO aparecen las carpetas vacías (database/schema, scripts, runbooks...), solo README.md como "untracked". Esto sorprende a casi todos los principiantes.

⚠ ERROR FRECUENTE — "¿Por qué no aparecen mis carpetas vacías en git status?"

Git no versiona carpetas, versiona archivos. Una carpeta completamente vacía no deja ningún rastro que Git pueda registrar.

Solución profesional: si necesitas conservar una carpeta vacía en el repositorio (por ejemplo runbooks/ antes de escribir el primer runbook), coloca dentro un archivo placeholder, normalmente llamado .gitkeep (convención, no es una palabra reservada de Git) o un README.md explicando el propósito de la carpeta.

Terminal — truco profesional para conservar carpetas vacías

```
touch runbooks/.gitkeep playbooks/.gitkeep database/schema/.gitkeep
database/migrations/.gitkeep database/rollback/.gitkeep database/tests/.gitkeep
scripts/.gitkeep
git status
```

Para ver la estructura en forma de árbol en Windows

Terminal — En Windows con git bash para ver la estructura de directorios con cmd o en bash con find.

```
cmd //c tree
find . -print | sed -e 's;[^\/*];| ;g'
```

Parte D — README y primer commit

Edita README.md con el siguiente contenido:

README.md

```
# Oracle Database Lab

Training repository for Oracle Database administration,
testing, change management and Git workflows.

Name: ESCRIBE_SIEMPRE_TU_NOMBRE_EN_CADA_README
Professor: Richard Aviles Lopez
```

Puedes editarlo con el block de notas. Si prefieres con VSCode: 1) abres la carpeta Oracle-database-lab, 2) abres el archivo Readme.md, copias el contenido, CTRL-S para grabar. Luego, puedes abrir una terminal git bash y observarás tu directorio actual con (master) al final.

Paso 1 — Observar el estado

Terminal

```
git config --global status.showUntrackedFiles all
git status
```

Git debería listar README.md (y los .gitkeep) como "Untracked files", en rojo. Untracked significa: Git ve el archivo en el disco, pero todavía no lo sigue.

Paso 2 — Mover a la Staging Area

Terminal

```
git add README.md
git status
```

Ahora README.md aparece en verde, bajo "Changes to be committed". Ha pasado del Working Directory a la Staging Area. Todavía no existe ningún commit.

Paso 3 — Revisar antes de confirmar (buena práctica)

Terminal — muestra exactamente qué vas a confirmar

```
git diff --staged
```

Paso 4 — Primer commit con Conventional Commits

Terminal

```
git commit -m "docs: add initial project documentation"
git status
git log
```

`git status` ahora debería decir "nothing to commit, working tree clean" para README.md. `git log` muestra el historial de commits en el repositorio, en este caso, tu primer commit, con su hash, autor, fecha y mensaje.

El código hash sirve para identificar un commit sin ambigüedad, volver a un punto exacto del historial y comparar versiones.

Conventional Commits — el estándar que usaremos todo el curso

La especificación Conventional Commits define la estructura:

Formato

```
<tipo>[ámbito opcional]: <descripción>
```

Tipo	Cuándo usarlo
feat	Añade una funcionalidad nueva
fix	Corrige un error
docs	Cambios solo en documentación
test	Añade o corrige tests
refactor	Cambia el código sin alterar el comportamiento
perf	Mejora de rendimiento
build / ci	Cambios en compilación o en la pipeline de integración continua
chore	Tareas de mantenimiento (dependencias, configuración menor)
security	Cambios relacionados con seguridad

⚠ ERROR FRECUENTE — Mensajes de commit sin significado

Mensajes como "cambios", "cosas", "final", "final definitivo", "prueba" o "asdf" son inútiles dentro de seis meses: nadie (ni tú mismo) sabrá qué ocurrió sin abrir el diff completo.

Compara: ``fix(db): correct customer foreign key`` te dice **QUÉ** cambió, **DÓNDE** (ámbito db) y de **QUÉ TIPO** es el cambio, en una sola línea.

✓ BUENA PRÁCTICA — One logical change per commit

Un commit debe representar un único cambio lógico. No mezcles "crear tabla + arreglar tests + actualizar documentación" en un solo commit: divide en varios commits. Esto facilita enormemente la revisión, el diagnóstico y el rollback futuro.

Parte E — Equivocarse deliberadamente (y corregirlo con seguridad)

Esta parte es intencionadamente un ejercicio de error controlado. Todo profesional comete estos fallos; lo importante es saber diagnosticarlos sin miedo.

Ejercicio 1 — El commit "a medias"

Crea un nuevo archivo de documentación pero olvida añadirlo antes de confirmar:

Terminal

```
echo "# Database schema notes" > docs/customer-schema.md
git commit -m "docs: add customer schema notes"
git status
git log --oneline
```

Resultado esperado: Git responde algo como "nothing added to commit but untracked files present" — el commit NO se realizó, porque el archivo nunca llegó a la Staging Area. Este es el error que casi todo principiante comete la primera semana: *crear que `git commit` incluye automáticamente los archivos nuevos.*

⚠ ERROR FRECUENTE — "Hice commit pero mi archivo nuevo no aparece en el historial"

``git commit`` únicamente confirma lo que está en la Staging Area.

Si el archivo es nuevo (untracked) y no hiciste ``git add``, el commit simplemente no lo incluye.

Diagnóstico:

- Ejecuta siempre ``git status`` inmediatamente antes de confirmar, y
- ``git diff --staged`` para ver exactamente qué vas a grabar.

Corrección:

Terminal

```
git add docs/customer-schema.md
git commit -m "docs: add customer schema notes"
git log --oneline
```

⚠ ERROR FRECUENTE — "Estoy perdido, no sé qué he commiteado ¿cómo puedo ver los commit que he realizado?"

```
`git log --online`
```

Muéstrame cada commit resumido en una sola línea de la rama actual.

Ejercicio 2 — El mensaje de commit con errata

Terminal — mensaje con errata

```
echo "Some content" >> docs/customer-schema.md
git add docs/customer-schema.md
git commit -m "dcos: fxi typo"
```

Si el commit AÚN NO se ha subido a GitHub (push), puedes corregir el último commit con --amend:

Terminal

```
git commit --amend -m "docs: fix typo in schema notes"
git log --online
```

⚠ ERROR FRECUENTE — Nunca hagas --amend sobre un commit ya subido (pushed)

`git commit --amend` reescribe el historial: el commit anterior desaparece y se crea uno nuevo con distinto hash. Si ese commit ya existía en el repositorio remoto y alguien más lo descargó, reescribirlo provoca divergencias e historiales incompatibles muy dolorosos de resolver en equipo.

Regla simple para principiantes:

--amend es seguro mientras el commit sea solo local.

En cuanto hagas `git push`, considera ese commit "público" e inmutable; para corregirlo, haz un commit nuevo.

Parte F — Leer el historial

Comandos de consulta más usados

git log	# historial completo y detallado
git log --oneline	# una línea por commit
git log --oneline --graph --all	# con gráfico de branches
git show <hash>	# detalle de un commit concreto
git diff	# cambios sin stage aún
git diff --staged	# cambios ya en la Staging Area

Acostúmbrate a `git log --oneline --graph --all` desde hoy: es el comando que usarás cientos de veces para entender "qué está pasando" en un repositorio con varias branches.

Parte G — Branches: crear, cambiar, comprobar que NO es una carpeta

Terminal — lista de branches (solo main)

```
git branch
```

Terminal

```
git switch -c feature/customer-search  
git branch  
ls -la
```

Comprueba con tus propios ojos: `ls -la` muestra EXACTAMENTE las mismas carpetas y archivos que antes. No ha aparecido ninguna carpeta feature/. La branch es un puntero dentro de .git, no una carpeta del disco.

Terminal — un commit exclusivo de esta branch

```
echo "Draft: customer search notes" > docs/customer-search.md  
git add docs/customer-search.md  
git commit -m "docs: draft customer search notes"  
git log --oneline --graph --all
```

¿a dónde apunta el HEAD? ¿En qué orden aparecen los commits? Responde:

Terminal — el contenido del disco cambia al cambiar de branch

```
git switch main  
ls docs/  
git switch feature/customer-search  
ls docs/
```

⚠ ERROR FRECUENTE — 'No existe main, me sale error fatal'

Si te sale error de que no existe main, seguro lo tienes como master. Puedes renombrar la rama con `git branch -m master main`.

Luego puedes cambiar la configuración con `git config --global init.defaultBranch main`

Aquí ocurre la revelación pedagógica: al volver a main, customer-search.md desaparece del disco (porque ese commit no existe en main); al volver a la branch, reaparece.

Git está reescribiendo el contenido del Working Directory según a qué commit apunta la branch activa — sin haber creado ninguna carpeta nueva.

✓ BUENA PRÁCTICA — checkout -b vs switch -c

Verás en tutoriales y en compañeros de trabajo el comando antiguo `git checkout -b nombre-branch`. Hace lo mismo que `git switch -c nombre-branch`. Son equivalentes para crear y cambiar a una rama nueva.

`switch` es el comando moderno, pensado específicamente para branches (más claro que `checkout`, que también sirve para restaurar archivos).

Ambos son válidos; en este curso usaremos `switch`, pero debes reconocer `checkout -b` cuando lo veas.

Terminal — eliminar una branch local (solo si ya está fusionada)

```
git branch -d nombre-de-una-branch-de-prueba
```

⚠ ERROR FRECUENTE — "Empecé a trabajar directamente en main sin querer"

Es el error más común de la primera semana: abrir el editor y modificar archivos estando en main, sin haber creado antes una branch.

Si aún no has hecho commit, puedes crear la branch ahora mismo con los cambios sin confirmar

Git los traslada contigo: `git switch -c feature/nombre` (los cambios sin commit no se pierden, viajan a la branch nueva).

Parte H — Provocar y resolver un conflicto

Vamos a simular a dos desarrolladores modificando la misma línea del mismo archivo. Trabajarás tú solo, pero interpretando dos roles distintos.

Paso 1 — Vuelve a main y crea una branch A

Terminal

```
git switch main  
git switch -c fix/readme-title
```

Edita la primera línea de README.md:

README.md — línea 1, versión A

```
# Oracle Database Lab (Training Edition)
```

Terminal

```
git status  
git add README.md  
git status  
git commit -m "docs: rename project title (training edition)"  
git status -c
```

Comenta lo que ha pasado con git status.

Paso 2 — Crea una branch B desde main y modifica la MISMA línea

Terminal

```
git switch main  
git switch -c fix/readme-subtitle
```

README.md — línea 1, versión B (distinta a la versión A)

```
# Oracle Database Lab – Academic Version
```

Terminal

```
git add README.md
git commit -m "docs: rename project title (academic version)"
git switch main
```

Paso 3 — Fusiona la primera branch (sin problema)

Terminal — merge limpio, main ya tiene la versión A

```
git merge fix/readme-title
```

Paso 4 — Fusiona la segunda branch (aquí aparece el conflicto)

Terminal

```
git merge fix/readme-subtitle
```

Salida esperada de Git

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Abre README.md. Verás algo así:

README.md con marcadores de conflicto

```
<<<<<< HEAD
# Oracle Database Lab (Training Edition)
=====
# Oracle Database Lab – Academic Version
>>>>>> fix/readme-subtitle
```

Marcador	Significado
<<<<<< HEAD	Empieza la versión que ya tenías en tu branch actual (main)
=====	Separador entre las dos versiones en conflicto
>>>>>> fix/readme-subtitle	Termina la versión que venía de la branch que estás fusionando

Paso 5 — Resolver

Decide (o combina) el contenido final, elimina TODOS los marcadores y deja el archivo limpio, por ejemplo:

README.md ya resuelto

```
# Oracle Database Lab (Training Edition – Academic Version)
```

Terminal — el commit de resolución cierra el merge

```
git status
git add README.md
git commit -m "merge: resolve README title conflict"
git log --oneline --graph --all
```

⚠ ERROR FRECUENTE — Dejar los marcadores <<<<<< sin quitar

El error más frecuente al resolver un conflicto es hacer `git add` y `git commit` sin haber borrado realmente los marcadores `<<<<<<`, `====`, `>>>>>>`. El commit se completa "sin errores" pero el archivo queda roto (código o documentación inválidos). Después de resolver, siempre revisa el archivo entero antes de confirmar, y si es código, ejecútalo o testéalo.

✓ BUENA PRÁCTICA — "Borro todo y empiezo de cero" (pánico ante un conflicto)

Un conflicto NO es un error grave ni una señal de que algo se ha roto: es Git pidiéndote una decisión que solo un humano puede tomar.

Si te bloqueas, te pones nervioso o te da ansiedad, siempre puedes abortar el merge sin perder nada y volver al estado anterior con `git merge --abort`, y reintentarlo con calma.

Si quieres empezar de nuevo esta parte borra las ramas desde main con `git branch -d fix/readme-title`
`git Branch -d fix/readme-subtitle`

Parte I — Conectar con GitHub

Paso 1 — Crear el repositorio remoto (en la web de GitHub)

1. Entra en github.com con tu cuenta.
2. Pulsa "New repository".
3. Nombre: oracle-database-lab.
4. NO marques "Add a README" (ya tenemos uno local) para evitar historiales que no encajan.
5. Crea el repositorio y copia la URL que te ofrece (HTTPS).

Paso 2 — Vincular tu repositorio local con el remoto

Terminal

```
git remote add origin https://github.com/tu-usuario/oracle-database-lab.git
git remote -v
git branch -M main
git push -u origin main
```

Cuando ejecutas el `git remote -v` te muestra los remotos configurados en tu repositorio Git. Te puede salir algo así:

origin <https://github.com/tuUsuario/oracle-database-lab.git> (fetch)

origin <https://github.com/tuUsuario/oracle-database-lab.git> (push)

El que indica fetch es el repositorio al que está vinculado. El que indica push, es hacia dónde haces git push. Como ambos apuntan a la misma URL, ese repo remoto es el mismo para descargar y subir cambios.

`-u` (--set-upstream) le dice a Git que, de ahora en adelante, esta branch local main está vinculada a origin/main.

Si te sale esta ventana, le das al botón azul.

Obtendrás esta salida del git push:

```
info: please complete authentication in your browser...
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 22 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (16/16), 1.55 KiB | 529.00 KiB/s, done.
Total 16 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/ravilesl/oracle-database-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Connect to GitHub

✕

GitHub
Sign in[Browser/Device](#) [Token](#)

Sign in with your browser

Sign in with a code

Don't have an account? [Sign up](#)

A partir de aquí, un simple `git push` o `git pull` (sin argumentos) ya sabrá con qué remoto y branch hablar.

Paso 3 — Subir el resto de branches (opcional para este laboratorio)

Terminal

```
git push -u origin fix/readme-title
git push -u origin fix/readme-subtitle
```

Paso 4 — Simular un cambio hecho "desde otro sitio" y traerlo (pull)

Edita un archivo directamente en la interfaz web de GitHub (por ejemplo, añade una línea a README.md desde el navegador) en la rama fix/readme-subtitle y confírmalo ahí. Después, en tu terminal:

Terminal — trae y fusiona automáticamente los cambios remotos

```
cat README.md
git switch fix/readme-subtitle
git pull
cat README.md
```

Git pull trae de github solo lo que corresponda a la rama en la que estas parado. No trae todo el main y todas las ramas.

Git vs GitHub

	Git	GitHub
Qué es	Un programa (sistema de control de versiones) que se ejecuta en tu máquina	Una plataforma web que aloja repositorios Git y añade colaboración
Dónde vive	En tu ordenador (y en el de cada colaborador)	En la nube
Funciona sin internet	Sí — init, add, commit, branch, merge, log son 100% locales	No — requiere conexión
Añade	Historial, branches, commits, merges	Pull Requests, Code Review, Issues, CI/CD, protección de branches

⚠ ERROR FRECUENTE — Errores típicos al conectar con GitHub

"fatal: remote origin already exists" → ya habías añadido un remoto antes llamado origin y lo tienes configurado; usa ``git remote set-url origin <url>`` para corregirlo en lugar de añadir otro.

"Support for password authentication was removed" → GitHub ya no acepta tu contraseña normal por HTTPS; necesitas un Personal Access Token o autenticación por SSH.

"! [rejected] main -> main (non-fast-forward)" → el remoto tiene commits que tú no tienes en local (por ejemplo, los hiciste desde la web). Solución: ``git pull`` primero traes lo que falta (o puedes hacer merge o rebase), que son forma de juntar ramas), resuelve conflictos si salen y después ``git push``. Merge une dos historiales y crea un commit de fusión si hace falta. Rebase reordena tus commits encima de otra base, pero no conviene usarlo en commits ya compartidos con otros.

5. Tabla consolidada de errores frecuentes

Resumen de referencia rápida para pegar en la pared del laboratorio o consultar durante cualquier práctica futura.

Error / síntoma	Causa habitual	Solución	Buena práctica relacionada
Hice commit y mi archivo nuevo no aparece	El archivo era "untracked" y no se hizo git add antes del commit	git add <archivo> y repetir el commit (o usar --amend si aún no hiciste push)	Ejecutar git status antes de cada commit
git status sigue mostrando el archivo como "modified" tras el commit	Se hizo otro cambio después del add, o solo se confirmó una versión parcial	git add de nuevo y nuevo commit	git diff --staged antes de confirmar
El mensaje de commit tiene una errata y no fue empujado (push)	Error de escritura al confirmar	git commit --amend -m "mensaje corregido"	Nunca --amend sobre commits ya subidos
"CONFLICT (content): Merge conflict"	Dos branches modificaron la misma línea del mismo archivo	Editar el archivo, eliminar marcadores <<<<<< =====>>>>>>, git add, git commit	Branches de vida corta y pull frecuente reducen los conflictos
El archivo queda con marcadores <<<<<< tras "resolver"	Se hizo add/commit sin revisar el contenido final	Reabrir el archivo, limpiar marcadores, volver a probar/testear, nuevo commit	Revisar el diff completo antes de confirmar una resolución
"non-fast-forward" / rechazo al hacer push	El remoto tiene commits que no están en tu copia local	git pull (fusiona) y después git push	Hacer pull al empezar cada sesión de trabajo
Carpetas vacías no aparecen en git status	Git no versiona carpetas, solo archivos	Añadir un archivo placeholder (.gitkeep, README.md)	Documentar el propósito de cada carpeta desde el inicio
Se subió sin querer un archivo con contraseñas o claves	No existía .gitignore antes del primer commit	Eliminar el secreto del historial (fuera del alcance de este lab) y rotar la credencial inmediatamente	Crear .gitignore y .env.example antes del primer commit; nunca commitar secretos
"fatal: remote origin already exists"	Ya se había vinculado un remoto anteriormente	git remote set-url origin <url>	git remote -v para comprobar antes de añadir
Se trabajó directamente sobre main por error	No se creó una branch antes de empezar	git switch -c feature/nombre (los cambios sin commit viajan con la branch nueva)	Crear siempre una branch antes de tocar código
git add . añadió archivos que no debían subirse	Uso ciego de "añadir todo" sin revisar	git restore --staged <archivo> para sacarlo de la Staging Area	Revisar git status/diff antes de un add masivo
Pánico y se borra la carpeta del proyecto ante un conflicto	Falta de confianza en que el conflicto es reversible	git merge --abort para volver al estado anterior sin pérdidas	Recordar: un conflicto es una pregunta, no una rotura

6. Buenas prácticas de la industria (resumen)

- Configura tu identidad (user.name / user.email) antes del primer commit.
- Crea una branch antes de empezar cualquier cambio; nunca trabajes directamente sobre main.
- Haz commits atómicos: un cambio lógico por commit.
- Escribe mensajes con Conventional Commits: tipo(ámbito): descripción clara.
- Revisa siempre git status y git diff --staged antes de confirmar.
- Crea .gitignore desde el primer commit; nunca subas contraseñas, claves ni tokens.
- Haz pull al empezar cada sesión de trabajo para minimizar conflictos.
- Ante un conflicto, léelo con calma: identifica HEAD (tu versión) frente a la branch entrante.
- Usa git log --oneline --graph --all para entender el estado real del repositorio.
- Evita --amend y cualquier reescritura de historial sobre commits ya subidos (push).

7. Preguntas de comprobación

Responde con tus propias palabras. El objetivo no es memorizar comandos, sino demostrar que entiendes qué ocurre en cada zona de Git.

1. ¿Cuál es la diferencia entre Working Directory, Staging Area y Local Repository? Da un ejemplo de un archivo pasando por las tres.
2. Si modificas un archivo pero no haces git add, ¿aparece ese cambio en tu próximo commit? Explica por qué.
3. ¿Por qué git status no mostraba las carpetas vacías que creaste en la Parte C? ¿Qué truco usamos para solucionarlo?
4. Explica con tus palabras qué es HEAD.
5. ¿Qué diferencia hay entre crear una branch con git switch -c y crear una carpeta nueva con mkdir? ¿Cómo lo comprobamos en la Parte G?
6. Durante el conflicto de la Parte H, ¿qué representaba el contenido entre <<<<<< HEAD y =====? ¿Y entre ===== y >>>>>>?
7. ¿Por qué NO se debe hacer git commit --amend sobre un commit que ya se subió con git push?
8. Si borras por accidente la carpeta .git de tu proyecto, ¿qué se pierde exactamente? ¿Se pierde también el código fuente que está en el disco?
9. Explica con tus propias palabras la diferencia entre Git y GitHub, sin usar la palabra "nube".
10. ¿Por qué no se debe subir un archivo .env con contraseñas reales a un repositorio, aunque el repositorio sea privado?
11. Un compañero te dice: "hice push y ahora GitHub me rechaza el segundo push con 'non-fast-forward'". ¿Qué ha ocurrido probablemente y qué comando ejecutarías primero?
12. ¿Qué tipo de Conventional Commit (feat, fix, docs, test...) usarías para: añadir un índice de rendimiento a una tabla, corregir una restricción mal definida, y actualizar el README?

8. Checklist de entrega

Para considerar el Laboratorio 1 completado, el repositorio en GitHub debe cumplir:

- El repositorio oracle-database-lab es visible en tu cuenta de GitHub.
- El historial (`git log --oneline --graph --all`, o la vista de "Network" en GitHub) muestra al menos: el commit inicial, los dos commits de la Parte E, los commits de las dos branches de la Parte H y el commit de resolución del merge.
- README.md contiene el título final ya resuelto, sin marcadores de conflicto.
- Existen las carpetas database/schema, database/migrations, database/rollback, database/tests, scripts, runbooks, playbooks y docs (con su .gitkeep o contenido).
- El repositorio remoto (origin) está correctamente vinculado: `git remote -v` muestra la URL de GitHub.
- Las Preguntas de comprobación (Sección 8) están respondidas por escrito en docs/lab1-respuestas.md y confirmadas mediante un commit docs: `add lab 1 answers`.

9. Rúbrica de evaluación

Criterio	Insuficiente	Adecuado	Excelente	Peso
Configuración e higiene del repositorio	Falta identidad configurada o estructura incompleta	Estructura completa, identidad correcta	Además usa .gitkeep con criterio y estructura impecable	15%
Calidad de los commits	Mensajes genéricos, commits mezclando varios cambios	Conventional Commits aplicado de forma básica	Commits atómicos, mensajes claros y consistentes en todo el historial	25%
Manejo de branches	No comprende que una branch no es una carpeta	Crea y cambia de branch correctamente	Explica con precisión el comportamiento de HEAD y del working tree al cambiar de branch	20%
Resolución del conflicto	No completa el conflicto o deja marcadores sin resolver	Resuelve el conflicto correctamente con ayuda	Resuelve el conflicto de forma autónoma y explica cada marcador	25%
Integración con GitHub	Repositorio remoto ausente o mal vinculado	Push y pull funcionando correctamente	Además gestiona errores de sincronización (non-fast-forward) sin ayuda	10%
Preguntas de comprobación y lecciones aprendidas	Respuestas vacías o incorrectas	Respuestas correctas pero superficiales	Respuestas con ejemplos propios y razonamiento claro	5%

10. Glosario rápido de este laboratorio

Término	Significado
Working Directory	La carpeta del proyecto tal como la ves y editas en tu disco
Staging Area	Zona intermedia donde preparas exactamente lo que irá en el próximo commit
Local Repository	El historial de commits guardado en la carpeta .git de tu máquina
Remote Repository	La copia del repositorio alojada en un servidor (en este lab, GitHub)
Commit	Una fotografía permanente y con mensaje de un conjunto de cambios
Branch	Una línea independiente de evolución del historial; no es una carpeta
HEAD	Puntero que indica en qué commit / branch te encuentras ahora mismo
Merge	Operación que combina el historial de dos branches
Conflict	Situación en la que Git no puede decidir automáticamente qué versión conservar
Push	Enviar tus commits locales al repositorio remoto
Pull	Traer y fusionar los commits del repositorio remoto a tu copia local
Conventional Commits	Especificación de formato para mensajes de commit: tipo(ámbito): descripción

11. Cierre del laboratorio

✓ BUENA PRÁCTICA — Frase para llevarse a casa

"Hoy no has aprendido comandos sueltos: has aprendido a moverte con seguridad entre cuatro zonas de trabajo, a comunicar tus cambios con mensajes claros y a resolver un conflicto sin pánico. Eso es exactamente lo que se espera de ti tu primera semana en un equipo real."

PRECAUCIÓN:

*No te confíes al avanzar,
que un cambio de máquina te puede complicar;
revisa remoto, rama y tu autenticación,
o Git te dará guerra sin explicación.*

En el Laboratorio 2 — GitHub Team Workflow, este mismo repositorio dejará de ser un ejercicio individual: introduciremos Issues, Pull Requests, Code Review formal y la protección de la branch main, de modo que ningún cambio pueda llegar a producción sin pasar por revisión y por la pipeline de integración continua.